

Privacy Preserving Collaborative IDS for Internet of vehicle networks

Click to edit Master subtitle style



#TITLE OF THE PROJECT , 01-07-2026

Content:

1. Architecture Pipeline

Click to edit Master subtitle style



Overall Architecture(phases):

1. Architecture Pipeline

Click to edit Master subtitle style

High-Level Architecture (The Pipeline)

Phase 1:

Goal: Establish a cryptographically authenticated state before federated training begins.

- **Step 1:** Load & Unify Datasets
- **Step 2:** Centralized Stratified Split (70% / 15% / 15%)
- **Step 3:** Dirichlet Non-IID Partitioning
- **Step 4:** Global Model Initialization
- **Step 5:** Secure Channel Setup
- **Step 6:** Genetic Algorithm (GA) Configuration
- **Step 7:** Encrypted Broadcast & Integrity Verification



Data Unification & Real-World Simulation

Datasets Unified: CAN-VTC, Car-Hacking, CICIDS 2017, and VeReMi.

- **Stratified Splitting:** * 70% Train | 15% Validation | 15% Test.
 - Statistics (z-score) fitted only on the training split to prevent data leakage.
- **Simulating the Fleet (Non-IID Distribution):**
 - Used Dirichlet allocation ($\alpha=0.3$).
 - Distributes training data unevenly across simulated clients (50, 100, or 200).
 - Creates realistic "class skew" so no two vehicles see the exact same distribution of threats.



Unified Dataset Adapter Interface:

```
def load_features_labels(root: Path, window: int, stride: int, max_rows:
Optional[int]) -> Tuple[np.ndarray, np.ndarray, Tuple[str, ...]]:
    feat_blocks, label_blocks = [], []
    for idx, cls in enumerate(CLASSES):
        # 1. Load the raw CAN-bus message attributes
        can_ids, dlc, payload = _parse_can_log(root / f"cleaned_{cls}_dataset.csv",
max_rows)

        # 2. Slice streams into sliding window features
        X = windows_from_stream(can_ids, dlc, payload, window, stride)
        y = np.full(X.shape[0], idx, dtype=np.int64)

        feat_blocks.append(X)
        label_blocks.append(y)

    # Returns unified shape: X: (N, 46), y: (N,)
    return np.concatenate(feat_blocks, axis=0), np.concatenate(label_blocks, axis=0),
CLASSES
```

Shows how raw data sources are read and converted into sliding windows of message streams.

For example, in `can_vtc_dataset.py` (and similarly for other logs), raw logs are parsed into numpy arrays, windowed, and labeled dynamically.



Unifying Heterogeneous Data to a Timeless 46-D Feature Vector

```
def extract_window_features(can_ids: np.ndarray, dlc: np.ndarray, payload:
np.ndarray) -> np.ndarray:
    """Unifies raw window logs into a timeless 46-dimensional feature vector."""
    feat = np.empty(46, dtype=np.float32)

    # 1. Payload byte distribution statistics (16 features)
    feat[0:8] = payload.mean(axis=0) # Mean of bytes 0..7
    feat[8:16] = payload.std(axis=0) # Std dev of bytes 0..7

    # 2. Length descriptor (2 features)
    feat[16] = dlc.mean()
    feat[17] = dlc.std()

    # 3. Frequency & Entropy descriptors (4 features)
    uniq, counts = np.unique(can_ids, return_counts=True)
    feat[18] = float(len(uniq)) / len(can_ids) # Unique CAN-ID ratio
    feat[19] = float(counts.max()) / len(can_ids) # Dominant ID frequency
    feat[45] = float((payload > 0).mean()) # Non-zero byte ratio

    # 4. Hashed CAN-ID Histogram distribution (24 features)
    buckets = np.zeros(24, dtype=np.float32)
    np.add.at(buckets, can_ids % 24, 1.0)
    feat[20:44] = buckets / buckets.sum()

    # 5. Shannon Entropy over payload bytes (1 feature)
    ent = 0.0
    for b in range(8):
        vals = (payload[:, b] * 255.0).astype(np.int32)
        _, c = np.unique(vals, return_counts=True)
        p = c.astype(np.float64) / c.sum()
        ent += float(-(p * np.log2(p + 1e-12)).sum())
    feat[44] = ent / 8.0

    return feat
```

Shows the mathematical/structural representation of the timeless CAN descriptors (from can_features.py) that enables dataset-agnostic training.

slots 0..7	payload byte means b0..b7	(normalised /255)
slots 8..15	payload byte stds b0..b7	
slots 16	DLC mean	
slots 17	DLC std	
slots 18	unique CAN-ID ratio	(count(unique IDs) / W)
slots 19	dominant-ID frequency	
slots 20..43	CAN-ID histogram over 24 hash buckets	
slots 44	mean Shannon entropy over the W payload byte distributions	
slots 45	non-zero payload byte ratio	

Total = 8 + 8 + 2 + 2 + 24 + 2 = 46 ✓

Normalization & Preprocessing

```
# 1. Stratified Partitioning
train_idx, val_idx, test_idx = _stratified_split(y, val_split, test_split, seed)

# 2. Percentile Clipping (fitted on training split only)
lo = np.quantile(X[train_idx], clip_low, axis=0)
hi = np.quantile(X[train_idx], clip_high, axis=0)
X = np.clip(X, lo, hi)

# 3. Z-score Standardisation (fitted on training split only)
mean = X[train_idx].mean(axis=0)
std = X[train_idx].std(axis=0) + 1e-6
X = (X - mean) / std

# 4. Outputs unified PyTorch datasets
return SplitResult(
    train=CanFeatureDataset(X[train_idx], y[train_idx]),
    val=CanFeatureDataset(X[val_idx], y[val_idx]),
    test=CanFeatureDataset(X[test_idx], y[test_idx]),
    ...
)
```

Shows how the splits are scaled identically to avoid data leakage (from build_dataset.py).



Dirichlet:

```
# Shard dataset indices across N clients using Dirichlet allocation
client_buckets = [[] for _ in range(n_clients)]

for c in classes:
    # 1. Get all sample indices for class 'c'
    idx_c = np.where(y == c)[0]
    rng.shuffle(idx_c)

    # 2. Draw proportions from Dirichlet distribution:  $p_c \sim \text{Dir}(\alpha * \mathbf{1}_N)$ 
    proportions = rng.dirichlet(alpha * np.ones(n_clients))

    # 3. Partition class 'c' indices based on these proportions
    splits = (np.cumsum(proportions) * len(idx_c)).astype(int)[:n_clients]
    chunks = np.split(idx_c, splits)

    # 4. Allocate chunks to corresponding clients
    for i, chunk in enumerate(chunks):
        if chunk.size:
            client_buckets[i].append(chunk)
```

It partitions a centralized dataset across N client vehicles such that each client receives a highly skewed, non-IID (non-independently and identically distributed) subset of data. This simulates realistic Internet of Vehicles (IoV) environments where different vehicles encounter different ratios of normal traffic and specific cyber-attacks.



Core Model & Cryptographic Security

The Model:

- **KANConvNet:** Global model initialized on the server with 145,668 parameters.
- Supports loading pre-trained weights with `strict=False` to adapt to different dataset class counts.

The Secure Channel:

- **Identity:** Long-term Ed25519 keys for mutual authentication.
- **Key Exchange:** ECDH-P256 generates fresh ephemeral keys.
- **Encryption:** FHE (functions left empty so FHE can be inserted into it)
- **Key-Derivation:** HKDF-SHA256 (HMAC-based Extract-and-Expand Key Derivation Function)



KANConvNet & Adapting Weights

```
class KANConvNet(nn.Module):
    def __init__(
        self,
        in_features: int = 46,
        n_classes: int = 4,
        layer_size: int = 128,
        dropout: float = 0.2,
        fourier_modes: int = 8,
        conv_channels: Tuple[int, int] = (32, 64),
        kernel_size: int = 5,
        use_conv_backbone: bool = True,
    ) -> None:
        super().__init__()
        # 1. KAN convolutional backbone (Fourier-basis activations)
        if use_conv_backbone:
            c1, c2 = conv_channels
            self.conv1 = KANConv1d(in_features, c1, kernel_size,
n_modes=fourier_modes)
            self.conv2 = KANConv1d(c1, c2, kernel_size, n_modes=fourier_modes)
            self.pool = nn.AdaptiveAvgPool1d(1)
            head_in = c2

        # 2. Fourier-basis Kolmogorov-Arnold Network Layer
        self.kan_in = FourierKANLinear(head_in, layer_size, n_modes=fourier_modes)
        self.act1 = KolmogorovActivation(layer_size)

        # 3. Hidden refinement (three Linear+ReLU blocks)
        self.hidden = nn.Sequential(...)
        self.act2 = KolmogorovActivation(layer_size)

        # 4. Final classification head
        self.head = nn.Linear(layer_size, n_classes)
```

Global model layout, which features a KAN convolutional backbone followed by a Fourier-basis KAN linear layer, hidden refinement blocks, and a final classification head:

Loading Weights with strict=False:

The server handles bootstrap weight adaptation to match the exact class structure of the target dataset:

```
# Load pre-trained weights (e.g. from an intrusion detection baseline)
ckpt = torch.load(pretrained_path, map_location="cpu", weights_only=True)
sd = ckpt.get("model_state_dict", ckpt)

# Allow shape mismatches on the final classification head (strict=False)
# to automatically adapt the model to different dataset class counts
missing, unexpected = model.load_state_dict(sd, strict=False)
```

ConvAutoEncoder parameters:

- Total Parameters: 45,998
- Encoder:
 1. Weights – 22,848
 2. Bias – 160
- Decoder:
 1. Weights – 22,848
 2. Bias - 142



The Secure Channel Cryptography

Identity: Long-term Ed25519 Mutual Authentication

Each entity holds a long-term asymmetric Ed25519 keypair for identity authentication and signing during handshakes

```
@dataclass
class IdentityKey:
    name: str
    private: ed25519.Ed25519PrivateKey
    public: ed25519.Ed25519PublicKey

    @classmethod
    def generate(cls, name: str) -> "IdentityKey":
        sk = ed25519.Ed25519PrivateKey.generate()
        return cls(name=name, private=sk, public=sk.public_key())
```

During the handshake, a party signs its ephemeral key plus the peer's public identity:

```
# Sign the ephemeral exchange key to authenticate identity
signature = local_id.private.sign(eph_pub_bytes + peer_id_pub_bytes)

# Peer verifies signature with long-term public key to prevent MITM attacks
pending.peer_id_pub.verify(peer_signature, peer_eph_pub_bytes + local_id_pub_bytes)
```

Key Exchange: Ephemeral ECDH-P256

Fresh, short-lived (ephemeral) elliptic curve keypairs are generated and swapped to derive a shared secret:

Key-Derivation: HKDF-SHA256 (HMAC-based Extract-and-Expand)

The shared secret is combined with the handshake transcript to safely derive a symmetric key:

Encryption: FHE adapter setup done so FHE part can be inserted here.

```
# 1. Generate fresh ephemeral ECDH-P256 keypair
eph = ec.generate_private_key(ec.SECP256R1())
eph_pub_bytes = eph.public_key().public_bytes(...)

# 2. Perform Elliptic Curve Diffie-Hellman (ECDH) key exchange
shared = pending.local_eph_priv.exchange(ec.ECDH(), peer_eph_pub)
```

```
# Deriving the 32-byte AES key using HKDF-SHA256
transcript = sorted_local_and_peer_ephemeral_pubkeys
aes_key = HKDF(
    algorithm=hashes.SHA256(),
    length=32,
    salt=transcript,
    info=b"FedIoV/v1/aes-256-gcm",
).derive(shared)
```



Optimization Setup (Genetic Algorithm)

Purpose: Prepares the system to automatically tune itself for different vehicular constraints.

- **Search Space Defined:** Includes variables like batch size, epochs, momentum, L2 regularization, dropout, and optimizer type.
- **Combinatorial Space:** 75,000 possible configurations.
- **Initial Population:** Sampled uniformly with a size of 50.
- **Evolution Parameters:** Locked to 20 generations with elitism set to 2.



Integrity Verification & Phase Boundaries

Broadcast Verification (Step 7):

- Server serializes and encrypts the global model state for each client.
- Clients decrypt and reconstruct the model locally.
- **Integrity Check:** The system compares the SHA-256 digest of the server's model against every client's reconstructed model to prove bit-perfect fidelity.

Excluded from Phase 1 by Design:

- CarNet dataset omitted per project requirements.
- Deferred to Phase 2: Local training rounds, TOPSIS reduction, Multi-Krum aggregation, GA fitness evaluation, and Differential Privacy noise.



Proof of Work & Smoke Test Results

System Sanity Check:

- **Execution Time:** Completed in ~4.51 seconds for 50 clients.
- **Cryptographic Verification:** 50/50 clients verified successfully.

1-Epoch Baseline Performance:

- **CAN-VTC:** 99.89% Accuracy (Model learns instantly).
- **Car-Hacking:** 94.96% Accuracy (Solid multi-class performance).
- **CICIDS-2017:** 94.33% Accuracy (High detection on ~400k rows, minimal false positives).
- **VeReMi:** 73.81% Accuracy (Reflects expected class imbalance; requires Phase 2 extended training to catch edge-case attacks).

